Latviz: An Interactive Visualization Tool for Lattice-Based Cryptographic Algorithms and Cryptanalysis

Jeslyn Theodora and Muhammad Imam Chaidar Mujaddid Al-Ghiffari

Faculty of Computer Science, University of Indonesia.

Abstract. Post-quantum cryptographic algorithms are gaining relevance as prevalent security measures are found vulnerable to quantum computers. Among them are the key encapsulation standard ML-KEM and the digital signature standard ML-DSA released by NIST (National Institute of Standards and Technology) in 2024. Understanding these schemes and the algorithms such as LLL and BKZ that are used to evaluate their security is vital to both the theoretical study and practical application of post-quantum cryptography. However despite their relevance, there are few resources dedicated to fully describing the algorithms behind them that are easily followed. This paper details the creation of Latviz, a digital visualization tool meant to help learners understand the entire process behind the lattice-based algorithms such as the ML-KEM, ML-DSA, LLL, and BKZ. It provides a complete interactive visual step-by-step breakdown of the algorithms, including supplementary explanations behind the relevant functions and variables. The tool was evaluated through a survey of 22 university students with varying degrees of cryptographic knowledge, focusing on how effective Latviz was at increasing user understanding of ML-KEM, ML-DSA, LLL, and BKZ. The results indicate that users generally perceived Latviz as effective in improving their understanding of all of them, with 77.3%, 81.8%, 72.7%, and 77.3% of respondents considering Latviz an effective learning tool for ML-KEM, ML-DSA, LLL, and BKZ respectively. These findings suggest that Latviz can serve as an effective tool for supporting learners in understanding ML-KEM, ML-DSA, LLL, and BKZ algorithms.

Keywords: Cryptography · Post-Quantum Cryptography · Algorithm Visualization · Visualization · ML-KEM · ML-DSA · LLL · BKZ · Lattice-based Cryptography

1 Introduction

Ever since quantum computing was first theorized in the early 1980s [6, 7], its potential capabilities have been continuously theorized and expanded upon. From improving computer simulation using quantum-based phenomena [13] to applying quantum theory to improve information security [8, 10], quantum computing has had profound implications on entire fields. Among them are its implications

on cryptography. In 1994, Peter Shor proposed a quantum algorithm that, given a sufficiently powerful quantum computer, might be used to break protocols like RSA [23]. Shor's algorithm and others like it significantly lower the difficulty of problems such as the integer factorization problem and discrete logarithm problem, which the most widely used public key algorithms base their difficulty on. This means that common security protocols are at risk of being broken sometime in the near future [20]. Because of this, cryptology experts around the world have looked into new protocols that might have the potential to stay secure despite the continued development into quantum computers.

NIST (National Institute of Standards and Technology) is one of the organizations at the forefront of the effort. In 2016, they put out a call for proposals that might be used in post-quantum cryptography. Eight years later, they released three standards based on those submissions, two of which are ML-KEM and ML-DSA, based on the Kyber and Dilithium algorithms respectively [22, 21]. ML-KEM is used for key encapsulation, while ML-DSA is used for digital signatures. Both fall under the umbrella of lattice-based cryptography.

The security of those algorithms and others like them are derived from the computational difficulty of problems defined on mathematical lattices. Evaluating the security of such lattice-based schemes requires the use of lattice basis reduction algorithms such as LLL (Lenstra-Lenstra-Lovász) and BKZ (Block Korkine-Zolotarev), which form the foundation of many practical analyses and cryptanalysis techniques.

However, despite their importance, there are a limited number of resources available for learning about them. Outside of NIST's official releases and lectures, most of resources for ML-KEM and ML-DSA come in the from of summaries of said releases and lectures, public implementations, and simplified diagrams or visualizations. The visualization and exploration tool pqc-vizz [16] for example, provides an interactive working implementation for ML-KEM and ML-DSA that describes the overall process of using them, but skips over most of their inner workings such as their background calculations. Daeukun Kang's ML-KEM from Scratch [17] details the entire process of ML-KEM as well as the context and mathematical grounding behind it, including a working Python implementation. However, it assumes that the user has a working understanding of coding; otherwise, it is nothing more than a particularly structured summary. Other resources are similarly lacking in that they do not offer anything beyond a summary of NIST's release or they simplify their explanations to only the outermost layers.

In the case of LLL and BKZ, most resources are solvers that only show the final result of the reduction process and in general, are not user friendly. The fplll library [12] can be used to to perform LLL and BKZ, but it only outputs the final result. While it also comes with several implementations, it lacks an actual user interface that would make it easy to use. lll-attack-runner [9] provides an interactive application that is able to run LLL and BKZ. However, lll-attack-runner lacks visualization (which is the most convenient way to visualize vectors[11]) and mathematical explanations of the steps of its LLL and BKZ solvers. SageMathCell [14] has an in-built function to solve LLL, but requires

wrapping input matrices using one of the coding languages available to the tool. While it is capable of showing every step of the LLL solving process, that requires the use of custom code that the user either has to actively look for or make themselves. None of these three resources has visualizations, after all.

Thus, this work aims to develop a visualization tool Latviz that allows users to view the complete process behind ML-KEM, ML-DSA, LLL, and BKZ algorithms in a way that is still digestible for most people. This is done by displaying the inner mechanisms of both algorithms such that users unfamiliar with cryptography can still follow the explanations, and as well as going through both processes in their entirety instead of focusing only on the inputs and outputs. To achieve this, the tool implements a set of features designed to improve accesibility while preserving technical accuracy.

- **Complete algorithmic breakdown**: Latviz provides a more in-depth breakdown of the ML-KEM and ML-DSA standards compared to existing resources. The variables and algorithms that are involved in the each stage of both standards are elaborated upon in a step-by-step manner, with the explanations simplified as to be digestible without complete understanding of the mathematical grounding required to otherwise understand it.
- **Interactive visual interface**: Merely viewing a visualization does not give significant advantages over conventional methods such as following lectures or reading in how they impact learning algorithms. The most successful uses of visualization tools for learning algorithms are when the tools are used as a vehicle for actively engaging in the process of learning said algorithms [15], such as in the form of analyses of algorithmic behavior through manipulating functions and variables in the Desmos Graphic Calculator where users can directly see how certain functions and variables affect graphs. For LLL and BKZ, this comes in the form of being able to manipulate the input matrices, δ, and β values and directly see how they affect the reduction process. While ML-KEM and ML-DSA doesn't allow for as much interactivity as most of the variables and functions are either set or completely randomized, Latviz provides interactivity in the form of the manipulation of parameter sets, stepping through operations, and the inspection of internal states.
- **Solver tracer**: Latviz allows for the tracing of the entirety of the reduction processes in LLL and BKZ. The user can go through each step and see the relevant variables for each step in textual form and on a 2D or 3D grid.

2 Related Works

2.1 Lattice-Based Cryptography

A lattice is a discrete subset of n-dimensional vectors of real numbers $\mathbb{R}^n$ formed by all integer linear combinations of a set of linearly independent basis vectors. Formally, given a basis $\mathbf{B} = \{\mathbf{b}_1, \ldots, \mathbf{b}_k\} \subset \mathbb{R}^n$, a lattice $\mathcal{L}$ is defined as:

$$\mathcal{L} = \left\{ \sum_{i=1}^{k} z_i \mathbf{b}_i \;\middle|\; z_i \in \mathbb{Z} \right\}$$

Lattices can be visualized as regularly spaced grids extending in multiple dimensions, but their structure becomes increasingly complex and difficult to visualize in high-dimensional spaces, and especially beyond the third dimension.

Lattice-based cryptography relies on the presumed hardness of certain computational problems defined over lattices. Two of the most important problems are the Shortest Vector Problem (SVP), which asks for the shortest non-zero vector in a lattice, and the Learning With Errors (LWE) problem, which involves solving systems of linear equations that have small random noise. As of now, no algorithms exist that can solve these problems efficiently even with quantum computers, and they are tentatively believed to be quantum resistant.

Practical lattice-based schemes often use structured variants such as Ring-LWE and Module-LWE (MLWE), which operate over polynomial rings. These variants significantly improve efficiency while retaining the core hardness properties. MLWE in particular provides a balance between performance and conservative security assumptions and forms the basis of several standardized post-quantum schemes, including ML-KEM and ML-DSA.

It should be noted that both ML-KEM and ML-DSA employ Number Theoretic Transform (NTT), a generalization of Discrete Fourier Transform (DFT) to finite fields, to accelerate polynomial multiplication. The NTT representation of a polynomial will be in hat notation(e.g., $\hat{A}$).

2.2 Learning with Errors

Learning with Errors is essential for modern lattice-based post-quantum criptographic algorithms, such as NIST's ML-KEM and ML-DSA [25]. Basically, this is a method to hide information (e.g plaintext) by injecting randomized noise (errors) into a system of linear equations which represents the public matrix, which would make the attackers struggle to get the actual pieces of the information from the plaintext. Solving an LWE problem means solving a set of linear equations with errors hidden in the equations. The errors are often tiny.

For example, given the public matrix A, a secret message or plaintext s, and a random small number to mess the equation e. Normally, without the latter, a cipher is usually defined as $C = As \pmod{q}$. With LWE however, the equation becomes $C = As + e \pmod{q}$. Without the small errors, the system of linear equations can be solved with Gaussian Elimination, making it easier to crack.

There are two types of LWE:

- Search LWE: This approach is to find s, the secret key from a noisy set of linear equations.
- Decision LWE: This approach is to distinguish an equation with LWE from a pure random noise. Usually, a real LWE set of equations would actually at first look random, but the errors gained from the solutions (lattice reduction algorithms) of these equations do usually cluster around the actual key.

The noise should be small enough that legitimate users can still recover the intended message. At the same time, the presence of this error prevents an at-

tacker from directly solving the underlying system of linear equations using standard techniques such as Gaussian elimination. The best known attacks typically involve constructing a lattice from the problem instance and applying lattice reduction algorithms to search for unusually short vectors, thereby reducing the attack to problems closely related to SVP (Shortest Vector Problem).

SVP itself is the problem of finding the shortest non-zero vector in a lattice [24]. For a vector in a lattice to be regarded as the SVP, a vector must be the shortest one (or one of the shortest ones), out of all vectors in the lattice, and it must be a nonzero vector. It is essential for PQC lattice reduction algorithms because it can help to solve the LWE problem used in PQC algorithms.

2.3 ML-KEM - FIPS 203

ML-KEM, also referred to as Module-Lattice-Based Key-Encapsulation Mechanism Standard, is a standard set by NIST's FIPS 203 publication that specifies a set of algorithms meant to be used to establish a shared private key between two parties. This standard was proposed in response to the potential unreliability of modern cryptographic key systems such as RSA in the face of quantum computers.

ML-KEM is a variant of the CRYSTALS-Kyber algorithm [1], which is itself based on the hardness of solving MLWE, which is based on the presumed hardness of certain computational problems in module lattices. It can be described as a generalization of the Learning with Errors (LWE) problem as previously described. Both LWE and MLWE are widely considered to be quantum resistant, in that there are no known quantum algorithms that can efficiently solve it.

ML-KEM and other KEMs (Key Encapsulation Mechanisms) typically consist of three main algorithms: a key generation, key encapsulation, and key decapsulation algorithm.

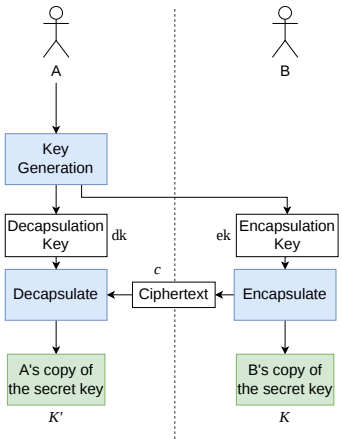


Fig. 1: Simplified key establishment process using KEM

As shown in Figure 1, KEMs are used to establish a shared private key between two parties. Party A begins by running a key generation algorithm that produces an encapsulation (public) key and decapsulation (private) key. When party B accepts the encapsulation key, they can run the encapsulation algorithm with it as an input, which produces a copy of the two parties' shared secret key K and its associated ciphertext c. Party B sends the ciphertext to party A, and party A runs the decapsulation algorithm using the decapsulation key and the ciphertext, producing their own copy K' of the shared secret key.

They also have multiple parameter sets, of which ML-KEM has three. Each set specifies the underlying ring dimension, module rank, noise distributions, and compression parameters. Increasing these parameters results in larger matrices, vectors, and noise levels, which enhances security at the cost of computational efficiency. In order of least to most secure and most to least efficient, they are ML-KEM-512, ML-KEM-768, and ML-KEM-1024.

Table 1: Parameter sets for ML-KEM (Kyber) security levels

	n	q	k	η_1	η_2	d_u	d_v	*Required RBG strength (bits)*
ML-KEM-512	256	3329	2	3	2	10	4	128
ML-KEM-768	256	3329	3	2	2	10	4	192
ML-KEM-1024	256	3329	4	2	2	11	5	256

Each parameter set is comprised of the five individual parameters k, η_1, η_2, d_u, and d_v, and two constants n and q.

- k determines the dimensions of the matrix A that appears in K-PKE.KeyGen and K-PKE.Encrypt, as well as the dimensions of vectors s and e in K-PKE.KeyGen and vectors y and e_1 in K-PKE.Encrypt. An increase in k comes with an increase in security level, key sizes, and ciphertext sizes, but also computational cost.
- η_1 specifies the distribution for generating vectors s and e in K-PKE.KeyGen and the vector y in K-PKE.Encrypt. Essentially, increasing η_1 produces larger noise terms and increases security level at the cost of efficiency.
- η_2 specifies the distribution for generating vectors error e_1 and e_2 in K-PKE.Encrypt.
- d_u and d_v are used as inputs and parameters for utility functions Compress, Decompress, ByteDecode, and ByteEncode in K-PKE.KeyGen and K-PKE.Encrypt.
- Constants n and q are the same between the three approved parameter sets. n denotes the degree of the polynomial ring for the scheme, while q is the modulus used for modular arithmetic.

2.3.1 ML-KEM Key Generation

The key generation algorithm ML-KEM.Keygen in ML-KEM produces an encapsulation key and decapsulation key using internally generated 32 byte seeds d and z. These keys are used in the encapsulation and decapsulation algorithms to establish a secure shared secret over a potentially insecure communication channel.

ML-KEM.KeyGen()

Generates an encapsulation key and a corresponding decapsulation key.

Output: encapsulation key ek $\in \mathbb{B}^{384k+32}$.
Output: decapsulation key dk $\in \mathbb{B}^{768k+96}$.

1: $d \xleftarrow{\$} \mathbb{B}^{32}$ ▷ d is 32 random bytes
2: $z \xleftarrow{\$} \mathbb{B}^{32}$ ▷ z is 32 random bytes
3: **if** $d ==$ NULL **or** $z ==$ NULL **then**
4: **return** $\perp$ ▷ return an error indication if random bit generation failed
5: **end if**
6: (ek, dk) $\leftarrow$ ML-KEM.KeyGen_internal(d, z) ▷ run internal key generation algorithm
7: **return** (ek, dk)

Fig. 2: ML-KEM.KeyGen() algorithm. Sourced from [22].

ML-KEM.KeyGen_internal(d, z)

Uses randomness to generate an encapsulation key and a corresponding decapsulation key.

Input: randomness $d \in \mathbb{B}^{32}$.
Input: randomness $z \in \mathbb{B}^{32}$.
Output: encapsulation key $\mathsf{ek} \in \mathbb{B}^{384k+32}$.
Output: decapsulation key $\mathsf{dk} \in \mathbb{B}^{768k+96}$.

1: $(\mathsf{ek}_{\mathsf{PKE}}, \mathsf{dk}_{\mathsf{PKE}}) \leftarrow$ K-PKE.KeyGen(d) ▷ run key generation for K-PKE
2: $\mathsf{ek} \leftarrow \mathsf{ek}_{\mathsf{PKE}}$ ▷ KEM encaps key is just the PKE encryption key
3: $\mathsf{dk} \leftarrow (\mathsf{dk}_{\mathsf{PKE}} \| \mathsf{ek} \| H(\mathsf{ek}) \| z)$ ▷ KEM decaps key includes PKE decryption key
4: **return** $(\mathsf{ek}, \mathsf{dk})$

Fig. 3: ML-KEM.KeyGen_internal() algorithm. Sourced from [22].

Assuming the generated random seeds d and z are valid, the seed d is used to run Kyber's key generation algorithm K-PKE.KeyGen, which outputs the encryption key and decryption key. The encapsulation key is the outputted encryption key of K-PKE. The decapsulation key consists of the outputted decryption key of K-PKE, the encapsulation key, a hash of the encapsulation key, and the random 32-byte value seed z.

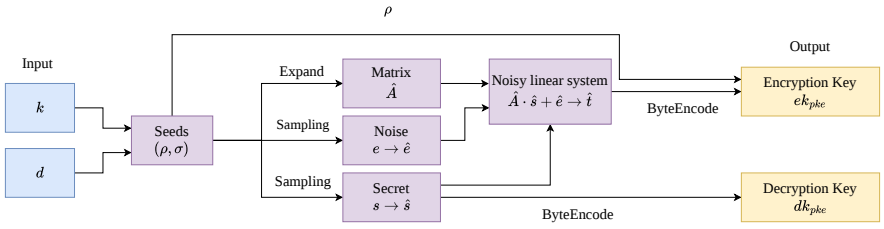


Fig. 4: Kyber Key Generation Algorithm K-PKE.KeyGen

K-PKE.KeyGen begins by generating 32 byte seeds ρ and σ from hashing the input d and k from the chosen parameter set. From the seeds, matrix A is determinalistically expanded from ρ and secret vectors s and noise e are sampled from centered binomial distributions using randomness expanded from σ. The three together form a noisy linear combination that computes the part t of the encryption key. The seed for matrix A is then appended to t to create the encryption key, while vector s makes up the decryption key. Both are encoded into the appropriate byte arrays, then output.

2.3.2 ML-KEM Encapsulation

The key encapsulation algorithm ML-KEM.Encaps accepts an encapsulation key and random value m as input and produces a shared key and its associated

ciphertext. The shared key serves as a secret value that can be used to secure subsequent communications, while the ciphertext is transmitted to a recipient party so that they can derive the same shared key.

ML-KEM.Encaps(ek)

Uses the encapsulation key to generate a shared secret key and an associated ciphertext.

Checked input: encapsulation key ek $\in \mathbb{B}^{384k+32}$.
Output: shared secret key $K \in \mathbb{B}^{32}$.
Output: ciphertext $c \in \mathbb{B}^{32(d_u k+d_v)}$.

1: $m \xleftarrow{\$} \mathbb{B}^{32}$ ▷ m is 32 random bytes
2: **if** $m ==$ NULL **then**
3: **return** $\perp$ ▷ return an error indication if random bit generation failed
4: **end if**
5: $(K, c) \leftarrow$ ML-KEM.Encaps_internal(ek, m) ▷ run internal encapsulation algorithm
6: **return** (K, c)

Fig. 5: ML-KEM.Encaps() algorithm. Sourced from [22].

ML-KEM.Encaps_internal(ek, m)

Uses the encapsulation key and randomness to generate a key and an associated ciphertext.

Input: encapsulation key ek $\in \mathbb{B}^{384k+32}$.
Input: randomness $m \in \mathbb{B}^{32}$.
Output: shared secret key $K \in \mathbb{B}^{32}$.
Output: ciphertext $c \in \mathbb{B}^{32(d_u k+d_v)}$.

1: $(K, r) \leftarrow$ G(m‖H(ek)) ▷ derive shared secret key K and randomness r
2: $c \leftarrow$ K-PKE.Encrypt(ek, m, r) ▷ encrypt m using K-PKE with randomness r
3: **return** (K, c)

Fig. 6: ML-KEM.Encaps_internal() algorithm. Sourced from [22].

A copy of shared secret K and the randomness r used during encryption are derived from the encapsulation key and m via hashing. The randomness r, the encapsulation key, and m are then used as input for Kyber's encryption algorithm, K-PKE-Encrypt, which will encrypt m into ciphertext.

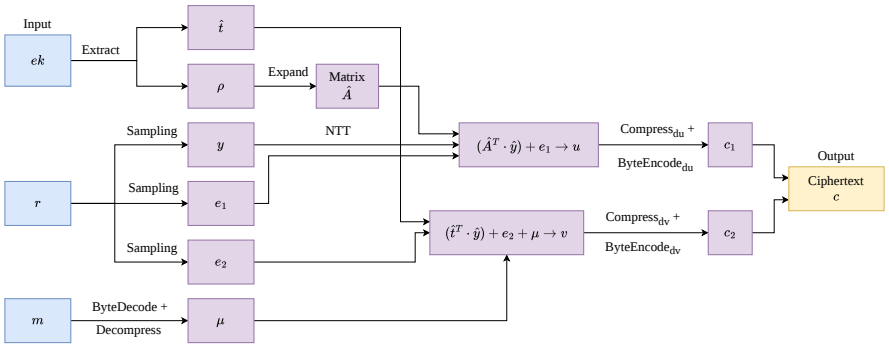


Fig. 7: Kyber Encryption Algorithm K-PKE.Encrypt

From the encryption key, $\hat{t}$ and seed ρ are extracted. Matrix $\hat{A}$ is then re-expanded from ρ. If extracted correctly, both $\hat{t}$ and $\hat{A}$ should be the same as they were in K-PKE.KeyGen.

Following that, the vector y and noise terms e_1 and e_2 are sampled from centered binomial distribution using pseudorandomness from input randomness r. The ciphertext is then computed in two parts through the noisy equations $(\hat{A}^T \cdot \hat{y}) + e_1$ and $(\hat{t}^T \cdot \hat{y}) + e_2$. The encoding μ of message m is added to the latter formula. The resulting pair (u, v) is compressed and converted into a byte array before being output as the ciphertext.

2.3.3 ML-KEM Decapsulation

The decapsulation algorithm ML-KEM.Decaps accepts a decapsulation key and ciphertext generated by ML-KEM as input and outputs a shared secret key. This allows the recipient to recover the same secret value generated by the sender, enabling both parties to securely communicate using a common secret.

ML-KEM.Decaps(dk, c)

Uses the decapsulation key to produce a shared secret key from a ciphertext.

Checked input: decapsulation key dk $\in \mathbb{B}^{768k+96}$.
Checked input: ciphertext $c \in \mathbb{B}^{32(d_u k + d_v)}$.
Output: shared secret key $K \in \mathbb{B}^{32}$.

1: $K' \leftarrow$ ML-KEM.Decaps_internal(dk, c) ▷ run internal decapsulation algorithm
2: **return** K'

Fig. 8: ML-KEM.Decaps() algorithm. Sourced from [22].

ML-KEM.Decaps_internal(dk, c)

Uses the decapsulation key to produce a shared secret key from a ciphertext.

Input: decapsulation key $\mathsf{dk} \in \mathbb{B}^{768k+96}$.
Input: ciphertext $c \in \mathbb{B}^{32(d_u k + d_v)}$.
Output: shared secret key $K \in \mathbb{B}^{32}$.

$\mathsf{dk}_{\mathsf{PKE}} \leftarrow \mathsf{dk}[0 : 384k]$ ▷ extract (from KEM decaps key) the PKE decryption key
$\mathsf{ek}_{\mathsf{PKE}} \leftarrow \mathsf{dk}[384k : 768k + 32]$ ▷ extract PKE encryption key
$h \leftarrow \mathsf{dk}[768k + 32 : 768k + 64]$ ▷ extract hash of PKE encryption key
$z \leftarrow \mathsf{dk}[768k + 64 : 768k + 96]$ ▷ extract implicit rejection value
$m' \leftarrow$ K-PKE.Decrypt$(\mathsf{dk}_{\mathsf{PKE}}, c)$ ▷ decrypt ciphertext
$(K', r') \leftarrow \mathsf{G}(m' \| h)$
$\bar{K} \leftarrow \mathsf{J}(z \| c)$
$c' \leftarrow$ K-PKE.Encrypt$(\mathsf{ek}_{\mathsf{PKE}}, m', r')$ ▷ re-encrypt using the derived randomness r'
if $c \neq c'$ **then**
    $K' \leftarrow \bar{K}$ ▷ if ciphertexts do not match, "implicitly reject"
end if
return K'

Fig. 9: ML-KEM.Decaps_internal() algorithm. Sourced from [22].

From the decapsulation key, the PKE encryption key ek_{PKE}, PKE decryption key dk_{PKE}, the hash value h of the PKE encryption key, and random value z are extracted. The ciphertext is decrypted through the K-PKE.Decrypt algorithm using the extracted decryption key, producing the plaintext message m'. The algorithm then re-encrypts the produced message is into a new output ciphertext c' along with computing candidate shared secret key K' as done in the encapsulation stage. If the newly produced ciphertext does not match the input ciphertext, the algorithm performs an 'implicit rejection', in which the value of K' is replaced with a hash of c appended to z. Otherwise, the decapsulation returns shared secret K' unchaged.

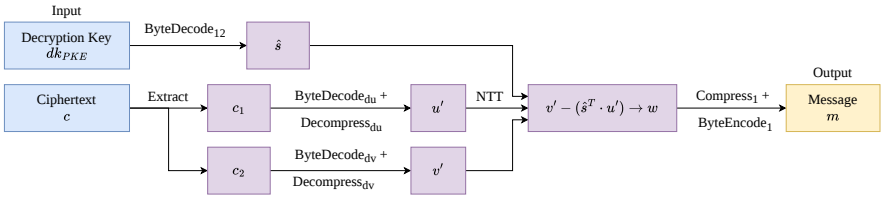


Fig. 10: Kyber Decryption Algorithm K-PKE.Decrypt

The K-PKE.Decrypt algorithm takes a decryption key and the corresponding ciphertext as inputs and extracts the secret variable s and parts c_1 and c_2 from them respectively. Part c_1 is converted into u' while c_2 is converted into v'. The true constant term v is computed through $s^t \cdot u'$. The resulting v is then subtracted from v', outputting w, which is decoded into a plaintext message m.

2.4 ML-DSA - FIPS 204

ML-DSA, also known as Module-Lattice-Based Digital Signature Standard, is a standard set by NIST's FIPS 204 publication that defines a method for digital signature generation and methods for the verification and validation of those signatures. The standard was proposed in response to the potential unreliability of modern digital signatures in the face of steady development of quantum computers.

ML-DSA is derived from the CRYSTALS-Dilithium algorithm [2], whose security is based on the computational hardness of the MLWE and a nonstandard variant of MSIS (Module-Short Integer Solution) called SelfTargetMSIS. Both MLWE and MSIS (which relies on the hardness of SVP) are considered quantum resistant, and ML-DSA inherets that quality.

ML-DSA and other digital signature schemes typically consist of three algorithms: a key generation algorithm, signing algorithm, and signature verifying algorithm.

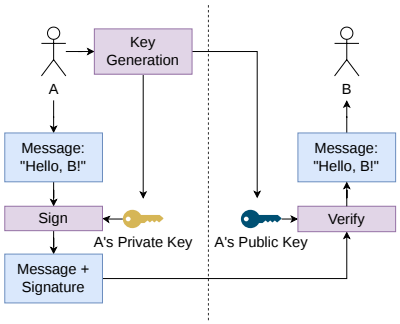


Fig. 11: Simplified digital signature scheme

As shown in Figure 11, digital signature schemes are used to provide assurance that a digital message or document is authentic. Party A begins by running a key generation algorithm that produces a public and private key. The private key is kept by party A, while the public key is given to party B. When party A sends a message or document to party B, they run the signing algorithm using their private key and the message, which produces a signature. Party B then can, upon receiving it, verify the message's authenicity using the corresponding public key.

ML-DSA also has three approved parameter sets. In order of least to most secure and most to least efficient, they are ML-DSA-44, ML-DSA-65, and ML-DSA-87.

Table 2: Parameter values for the ML-DSA parameter sets

	q	ζ	d	τ	λ	γ_1	γ_2	(k, ℓ)	η	β	ω
ML-DSA-44	8380417	1753	13	39	128	2^{17}	$\frac{q-1}{88}$	$(4, 4)$	2	78	80
ML-DSA-65	8380417	1753	13	49	192	2^{19}	$\frac{q-1}{32}$	$(6, 5)$	4	196	55
ML-DSA-87	8380417	1753	13	60	256	2^{19}	$\frac{q-1}{32}$	$(8, 7)$	2	120	75

Each parameter set is comprised of the variables τ, λ, γ_1, γ_2, (k, ℓ), η, β, and ω, along with the constants q, ζ, and d.

- τ determines the number of ± 1's in polynomial c in the signing algorithm.
- λ determines the collision strength of $\tilde{c}$ by controlling the output length of the hash function that produces it.
- γ_1 determines the coefficent range of polynomial vector y in the signing algorithm.
- γ_2 determines the low-order rounding range used in the decomposition and hint-generation procedures during signing and verification.
- (k, ℓ) determine the dimensions of matrix A and the sizes of vectors used throughout the scheme. Larger values increase security at the cost of larger signatures and higher computational costs.
- η determines the coefficient range of the secret vectors s_1 and s_2. The coefficients are sampled from a small bounded distribution to ensure the resulting lattice vectors remain short.
- β defines the rejection bound used during signing. It limits the allowable size of coefficients in intermediate values to prevent leakage of secret information.
- ω determines the maximum number of nonzero coefficients allowed in the hint vector h. This ensures that the hint remains sparse and compact.
- q is the modulus used for all polynomial coefficient arithmetic.
- ζ is a primitive 512th root of unity in $\mathbb{Z}_q$ used in the Number Theoretic Transform (NTT).
- d determines the number of low-order bits discarded during the `Power2Round` decomposition procedure used in public key compression.

2.4.1 ML-DSA Key Generation

The key generation algorithm ML-DSA.KeyGen chooses a random 32 byte seed ξ and uses it to generate a public key and private key. The public key can be shared with others and is used to verify signatures, while the private key is kept secret and is used to generate signatures. The seed itself is generated using an approved RBG (Random Bit Generator) such as described by Barker et al [5, 3, 4].

ML-DSA.KeyGen()

Generates a public-private key pair.

Output: Public key $pk \in \mathbb{B}^{32+32k(\text{bitlen }(q-1)-d)}$
and private key $sk \in \mathbb{B}^{32+32+64+32\cdot((\ell+k)\cdot\text{bitlen }(2\eta)+dk)}$.

1: $\xi \leftarrow \mathbb{B}^{32}$ ▷ choose random seed
2: **if** $\xi =$ NULL **then**
3: **return** $\perp$ ▷ return an error indication if random bit generation failed
4: **end if**
5: **return** ML-DSA.KeyGen_internal (ξ)

Fig. 12: ML-DSA.KeyGen() algorithm. Sourced from [21].

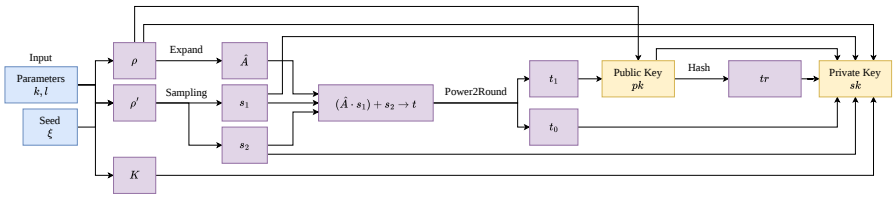


Fig. 13: ML-DSA Internal Keygen Algorithm ML-DSA.KeyGen_internal

The seed ξ is expanded into 32 byte random seed ρ, 64 byte random seed ρ', and 32 byte random seed K. Seed ρ is expanded into matrix A through sampling, and secret polynomial vectors s_1 and s_2 are sampled using seed ρ'. The three together compute part t of the public key, of which is decomposed into its higher and lower order components, t_1 and t_0 for the purposes of compression. The public key is a byte encoding of seed ρ and t_1. The private key is a byte encoding of seed ρ, seed K for use in signing, a hash tr of the public key, secret vectors s_1 and s_2, and t_0.

2.4.2 ML-DSA Signing

The signing algorithm ML-DSA.Sign takes a private key, a formatted message, and a context string (a byte string of 255 or fewer bytes) as input and outputs a signature encoded as a byte string. This signature serves as proof that the message was created by someone with the specific input private key, and that the message has not been altered since it was signed.

ML-DSA.Sign(sk, M, ctx)

Generates an ML-DSA signature.

Input: Private key $sk \in \mathbb{B}^{32+32+64+32\cdot((\ell+k)\cdot \text{bitlen }(2\eta)+dk)}$, message $M \in \{0,1\}^*$, context string ctx (a byte string of 255 or fewer bytes).
Output: Signature $\sigma \in \mathbb{B}^{\lambda/4+\ell\cdot 32\cdot(1+\text{bitlen }(\gamma_1-1))+\omega+k}$.

1: **if** $|ctx| > 255$ **then**
2: **return** $\perp$ ▷ return an error indication if the context string is too long
3: **end if**
4:
5: $rnd \leftarrow \mathbb{B}^{32}$ ▷ for the optional deterministic variant, substitute $rnd \leftarrow \{0\}^{32}$
6: **if** $rnd =$ NULL **then**
7: **return** $\perp$ ▷ return an error indication if random bit generation failed
8: **end if**
9:
10: $M' \leftarrow$ BytesToBits(IntegerToBytes$(0,1) \parallel$ IntegerToBytes$(|ctx|,1) \parallel ctx) \parallel M$
11: $\sigma \leftarrow$ ML-DSA.Sign_internal(sk, M', rnd)
12: **return** σ

Fig. 14: ML-DSA.Sign() algorithm. Sourced from [21].

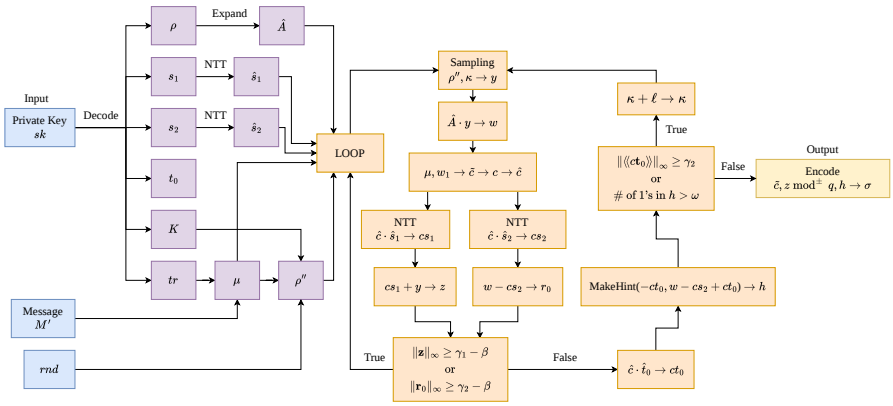


Fig. 15: ML-DSA Internal Signing Algorithm ML-DSA.Sign_internal

From the private key, the signer extracts the following: public random seed ρ, private random seed K, the hash of the public key tr, secret polynomial vectors s_1 and s_2, and lower order component of part t of the public key t_0. Seed ρ is then reexpanded into matrix A as in the key generation algorithm. Message M is concatenated to tr and hashed into 64 byte message representative μ. A 64 byte seed ρ'' is then produced for private randomness during each signing operation by hashing into 64 bytes a concatenation of K, random 32 byte string value rnd, and μ.

Following that, a rejection sampling loop that produces a signature is repeated until a valid one is created, which is then encoded as a byte string and output. The main computations involved in the loop are as follows:

- Generate vector y from using the ExpandMask function with ρ and counter κ as inputs.
- Commitment w_1 is computed by taking the higher order component of w, computed through $A \cdot y$.
- w_1 and μ are concatenated and hashed to produce the commitment hash $\tilde{c}$. $\tilde{c}$ is then used to pseudorandomly sample a polynomial c that has coefficients $\{-1, 0, 1\}$ and Hamming weight τ.
- Response $z = y + cs_1$ is computed. r_0 is also computed by taking the lower order components of $w - cs_2$. Both are used in validity checks that, if failed, restart the loop.
- If the validity checks pass, hint polynomial h is computed using the MakeHint function, which will allow the later verifier to reconstruct w_1 and other components of the signature correctly. After running more checks to see whether the coefficients in ct_0 are within a certain boundary and that the number of 1's in h does not exceed ω, assuming both checks pass, the final signature consisting of a byte encoding of commitment hash $\tilde{c}$, the response z, and the hint h will be output.

2.4.3 ML-DSA Verification

The verification algorithm ML-DSA.Verify takes a public key, a message, a signature, and a context string as input and outputs a boolean value that returns true if the signature is valid with respect to the message and public key and false if it is invalid. This means that anyone with access to the public key is able to verify the authenticity and integrity of a signed message.

ML-DSA.Verify(pk, M, σ, ctx)

Verifies a signature σ for a message M.

Input: Public key $pk \in \mathbb{B}^{32+32k(\text{bitlen }(q-1)-d)}$, message $M \in \{0,1\}^*$,
signature $\sigma \in \mathbb{B}^{\lambda/4+\ell\cdot 32\cdot(1+\text{bitlen }(\gamma_1-1))+\omega+k}$,
context string ctx (a byte string of 255 or fewer bytes).
Output: Boolean.

1: **if** $|ctx| > 255$ **then**
2: **return** $\perp$ ▷ return an error indication if the context string is too long
3: **end if**
4:
5: $M' \leftarrow$ BytesToBits(IntegerToBytes$(0,1)$ ‖ IntegerToBytes$(|ctx|,1)$ ‖ ctx) ‖ M
6: **return** ML-DSA.Verify_internal(pk, M', σ)

Fig. 16: ML-DSA.Verify() algorithm. Sourced from [21].

The verifier starts by extracting public random seed ρ and vector t_1 from the public key, then the commitment hash $\tilde{c}$, response z, and hint h from the signature. h is checked to see if it was encoded properly; if not, then the algorithm returns false. Otherwise, matrix A is expanded from ρ, tr is hashed from the public key and concatenated with the message input M', then hashed again to create message representation μ. The verifier's challenge c is computed from $\tilde{c}$ as in the signing algorithm.

The verifier then attempts to reconstruct the signer's commitment w_1'. In the verifier algorithm, w_1' requires w'_{approx}, which is computed through $A{\cdot}z{-}c{\cdot}(t_1{\cdot}2^d)$. w_1 is reconstructed by running the algorithm UseHint, which takes w'_{approx} and recovers the high bits of the w_1 calculated during signing using h. The result is concatenated to μ and hashed to create $\tilde{c}'$.

Finally, the verifier checks that response z is valid (by checking whether all coefficients in z are smaller than bound $\gamma_1 - \beta$) and that $\tilde{c}$ matches with $\tilde{c}'$. If both checks succeed, the verifier returns true. Otherwise, it returns false.

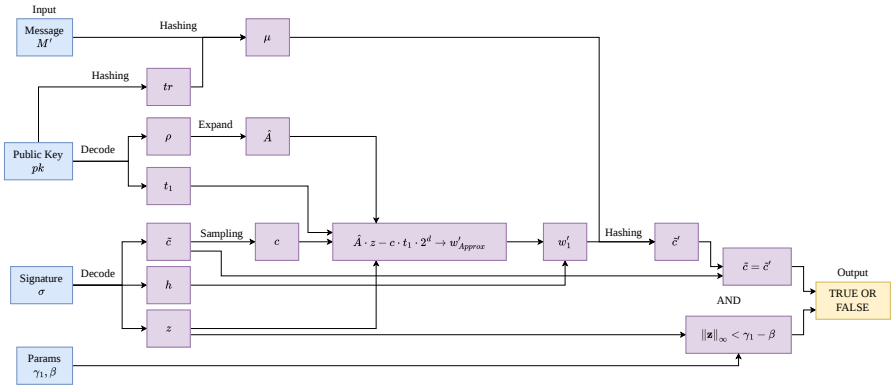


Fig. 17: ML-DSA Internal Verification Algorithm ML-DSA.Verify_internal

2.5 LLL

The LLL (Lenstra-Lenstra-Lovasz) cryptanalysis algorithm is a lattice reduction algorithm developed by Arjen Lenstra, Hendrik Lenstra, and Laszlo Lovasz in 1982. Although LLL is now largely ineffective for reducing lattices in many PQC algorithms such as ML-KEM and ML-DSA, the it is the foundation of modern lattice reduction cryptanalysis. For example, it serves as the foundation of BKZ, the upscaled version of LLL, by having several LLL processes executed for a single BKZ process.

In the Figure 18, we can see how the algorithm works. The inputs are in the form of matrices, or a set linear equations. For a single LLL process, the input

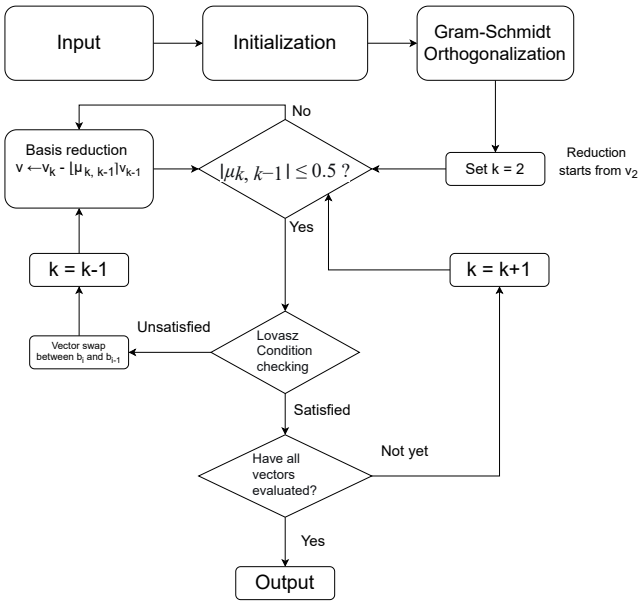


Fig. 18: The LLL algorithm

is in the form of a lattice basis L, which is in the form of a set of basis vectors $\{v_1, v_2, ...v_n\}$. Every vector within the set will be iteratively processed by Gram-Schmidt orthogonalization, then reduction or swap, and then examined whether the basis at the index fulfills the Lovasz Condition or not. The iteration process, however, is controlled by a pointer variable k which starts from 2nd index to the n-th index (the last vector). In other words, the first vector only acts as a projection reference for Gram-Schmidt orthogonalization (it only acts as the initial orthogonal basis ($\mathbf{v}_1^* = \mathbf{v}_1$)) and size reduction processes of subsequent vectors, but the first vector is never individually subjected to the vector swap mechanism (unless during the first iteration, the second vector fails to meet the Lovász Condition checking) or Lovász Condition checking. After all subsequent vectors have gone through these processes, the loop ends and the algorithm stops.

Unless the initial lattice basis is already a highly orthogonal basis, we will obtain a more orthogonalized and shorter version of the basis. Thus, with a basis that is more orthogonal and shorter, we would have it easier for recovering the LWE small errors hidden in the linear equations that form the public matrix we want to crack.

As an important note, the δ value used in Lovász Condition ranges from 0.25 as the minimum value to 0.99 as the maximum value. A higher δ yields higher quality outputs at the expense of more iterations, and a lower δ leads to fewer iterations at the expense of lower quality outputs. In other words, a higher δ makes the recovery of the LWE errors easier.

Below is the explanation of the LLL steps :

1. **Initialization.**

$$\mathbf{L} = \{v_1, v_2, ...v_n\}$$

L is a lattice basis to be reduced, with n vectors within it. L contains $\{v_1, v_2, ...v_n\}$. Assume L is a "bad" basis, then the goal is to make it more orthogonal and shorter.

2. **Gram-Schmidt orthogonalization.**.

$$\mathbf{v}_k^* = \mathbf{v}_k - \sum_{j=1}^{k-1} \frac{\langle \mathbf{v}_k, \mathbf{v}_j^* \rangle}{\|\mathbf{v}_j^*\|^2} \mathbf{v}_j^*$$

Right after the initialization, the algorithm processes the basis vectors sequentially. At the global scale (or the outer loop) in L, the algorithm performs Gram-Schmidt orthogonalization (GSO) for all vectors in L, then produces the initial orthogonalized $\mathbf{v}_k^*$ and their corresponding projection coefficients $\mu_{k,j}$, with $1 \leq j < k \leq n$ before the main loop execution begins. Notice that for the first vector ($k = 1$), no projection is needed since it has no preceding vectors. Hence, $\mathbf{v}_1^* = \mathbf{v}_1$.

3. **Main loop.** After the global Gram-Schmidt orthogonalization process, the index pointer k points at the second vector from the left (the vector of v_2), thus the main loop begins with the first iteration at $k = 2$.

4. **Basis reduction loop.** The algorithm checks the size of the vector, and then performs a size reduction of the vector to ensure that the length of the projection coefficients satisfies:

$$|\mu_{k,j}| \leq 0.5$$

. To satisfy the condition, the algorithm uses an internal index pointer, j, whose sole purpose is to become the backward iterator in this basis reduction loop. This loop starts from $j = k - 1$ and continues until 1. This ensures that the current target vector $\mathbf{v}_k$ is systematically reduced against all of its preceding vectors in the basis. The vector $\mathbf{v}_k$ is then iteratively updated using the following formula :

$$\mathbf{v}_k \leftarrow \mathbf{v}_k - \lfloor \mu_{k,j} \rceil \mathbf{v}_j$$

. Concurrently, to maintain mathematical consistency with the modified vector components, the algorithm must update $\mathbf{v}_k^*$ and the corresponding projection coefficients $\mu_{k,j}$ by fully recomputing the entire Gram-Schmidt orthogonalization sequence to get the updated $\mathbf{v}_k^*$ and $\mu_{k,j}$ values whenever $\mathbf{v}_k$ is modified inside this basis reduction loop. The basis reduction loop continues until the backward iterator j has fully decreased past 1, signifying that the $\mathbf{v}_k$ has been checked and reduced against every preceding vector in the basis L. Once the basis reduction loop ends, the updated vector is then subjected to the core step of the LLL algorithm - the Lovász Condition checking.

5. **Lovasz Condition checking.**

$$\|\mathbf{v}_k^*\|^2 \geq \left(\delta - \mu_{k,k-1}^2\right) \|\mathbf{v}_{k-1}^*\|^2$$

Using the updated $\mathbf{v}_k^*$ obtained from the preceding basis reduction loop, the algorithm evaluates whether the current vector ordering satisfies the Lovász Condition or not. Should the basis at the current iteration satisfies the Lovasz Condition, it will proceed to check whether all vectors have been examined. If that's the case ($k >= n$), then the main loop terminates, signifying that the entire basis has been successfully reduced, and followed immediately by the termination of the algorithm altogether. If not, then the iteration continues and k will point to the next vector ($k \leftarrow k + 1$), continuing the main loop until all vectors in the basis L have been fully processed. However, if the basis fails to satisfy the Lovasz Condition at the current iteration, a vector swap mechanism is executed and followed by a re-evaluation of the basis sequence.

6. **Vector swap.** If the current basis configuration does not satisfy the Lovász Condition at index k, it implies that the vector $\mathbf{v}_k$ has a significantly shorter orthogonal projection compared to its predecessor, $\mathbf{v}_{k-1}$, which makes the ordering of the vectors within L deficient. To fix this ordering deficiency, a vector swap mechanism must be executed. The order of the two adjacent vectors within the basis is interchanged as follows:

$$\mathbf{v}_{k-1} \leftrightarrow \mathbf{v}_k \tag{1}$$

Since this swap alters the geometric orientation and structural sequence of L, the previously calculated Gram-Schmidt vectors and coefficients become invalid from index $k-1$ onwards. Consequently, the algorithm retreats its main loop iteration counter using the following step :

$$k \leftarrow \max(k-1, 2) \tag{2}$$

This repositioning ensures that the counter shifts backward by one step, forcing the algorithm to recompute the local Gram-Schmidt orthogonalization and re-apply the basis reduction at this lower index before the Lovász Condition can be re-evaluated. As the important note, the max function serves as a logical guardrail to prevent k from accessing indices lower than 2.

7. **Output**. After all of the vectors have been examined and L meets the Lovasz Condition requirement, the algorithm terminates and returns a more orthogonalized and shortened version of L. This reduced lattice basis is more useful for recovering hidden LWE errors embedded within the linear equations.

2.6 BKZ

The BKZ (Block Korkine-Zolotarev) lattice reduction algorithm was first proposed by Clauss-Peter Schnorr in 1987, and in 1994 he and Martin Euchner published the journal of the BKZ algorithm [26]. This lattice reduction algorithm, while based on LLL, is much more stronger compared to LLL. Nowadays,

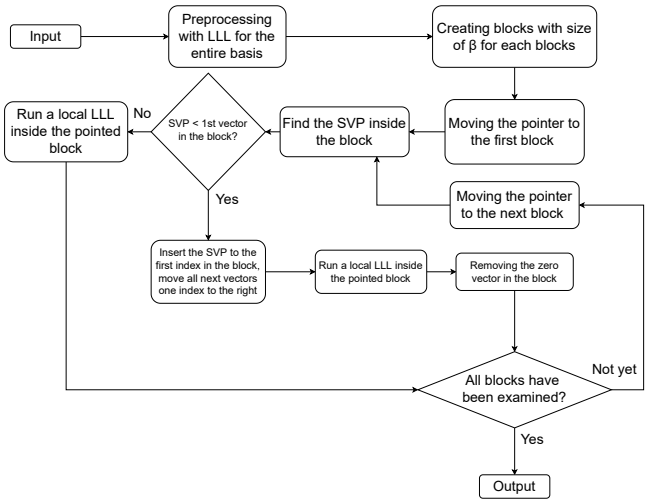


Fig. 19: The BKZ algorithm

the usage of BKZ in cryptanalysis is common, as the algorithm becoming increasingly important due to the emergence of the lattice-based post-quantum cryptographic algorithms [18].

Like in the LLL algorithm, the goal is to make L more orthogonal and shorter. The steps of the BKZ algorithm are explained below :

1. **Initialization and the first LLL.** The first step in BKZ algorithm is to run an LLL process, symbolized with

$$\mathbf{LLL}(\mathbf{L}, \delta)$$

on n vectors within $\{v_1, v_2, ...v_n\}$. As a side note, the δ-value used in BKZ for all LLL executions within a BKZ algorithm execution is the same, and normally the δ value used is 0.99. This is done to ensure a more orthogonalized and shorter outputs for each LLL execution and the final output of the BKZ execution as well.
2. **Main iteration.** After the first global LLL process has been completed, the algorithm begins the main iteration. While the LLL process iteratively processes the vectors one-by-one across the entire basis (for a single iteration in LLL, only one vector is reduced), BKZ processes vectors within localized regions of the basis called blocks. Think of it as a pointer that points β neighbouring vectors for each iteration, and performs SVP searching, vector swapping, and localized LLL, all exclusively for all vectors that are currently pointed. After the localized LLL is done, the pointer shifts just one vector to the right. So in other words, the block structure in BKZ behaves more like a sliding window with a fixed block size β that traverses the lattice basis from left to the right, and performs SVP enumeration, vector swapping, and

LLL only for all vectors within the block. For each single iteration, the BKZ algorithm selects a local block from the basis:

$$\mathbf{L}_{[i,h]} = \{v_i, v_{i+1}, ...v_h\}$$

in which

$$\mathbf{h} = \mathbf{max}(i + \beta - 1, n)$$

. Here, i is the starting index and h last index of the current block, while β is the fixed block size of the entire BKZ algorithm. For example, we have $\mathbf{L} = \{v_1, v_2, v_3, v_4, v_5\}$ with $\beta = 3$. The first block would be $\mathbf{L}_{[1,3]} = \{v_1, v_2, v_3\}$ and after the first iteration ends, the next iteration's block would be $\mathbf{L}_{[2,4]} = \{v_2, v_3, v_4\}$. As we see,the "sliding window" only moves by one vector to the left, which is why the BKZ's blocks are overlapping to each other. As an important note, since β-value is the symbolization of the block size, it means that the β-value determines how many neighboring vectors are included inside the current block during each iteration, and as such, the selection of the β-value has a big role on determining the quality of the BKZ's output. A smaller β produces smaller blocks, which means it produces a faster execution of the algorithm with the cost of the BKZ's output, while a bigger β resulted in slower execution but with better output.

3. **Localized SVP enumeration and LLL.** After the block formation, the BKZ algorithm searches the shortest non-zero vector (SVP) inside the current block. Shall the SVP candidate v is deemed to be shorter enough compared to the currently examined basis vector (as mathematically stated below) :

$$||v|| < \beta||b_i||$$

, then the SVP candidate v is deemed to be an improvement to the basis structure. Thus, the candidate v is inserted into the basis at the current block position $(v, b_i, b_{i+1}..., b_h)$. Also usually known as nontrivial insertion. Should the condition $||v|| \geq \beta||b_i||$ happens, then no insertion would be done, since the algorithm doesn't consider this case as an impactful improvement of the basis quality. After the local SVP enumeration, a localized LLL process is executed, which only involves all vectors within the current block.
4. **Termination and the resulted output.** Once all blocks within the lattice basis have been examined and processed, and no further nontrivial insertion can be executed, the BKZ algorithm terminates. Like the LLL algorithm, it would produce a shorter and more orthogonalized lattice basis compared to the initial basis.

Because the BKZ algorithm is formed from a global LLL, followed by the execution of localized LLL reductions and localized SVP enumerations in each of its blocks, the quality of a BKZ output is generally better (the output is expected to be much shorter and much more orthogonalized) than the output produced from a single LLL process. Thus, recovering LWE errors hidden in the system of linear equations within a lattice-based post-quantum cryptographic

system is generally better be done with BKZ. The only downside from a BKZ execution compared with LLL is the computational cost, since a single BKZ process involves several LLL processes.

3 Latviz

The visualization tool Latviz is a web application that provides visualizations for ML-KEM and ML-DSA, meant to help users more easily understand the two through interactive visualizations.

3.1 Architecture

Latviz is built using Next.js, a fullstack React-based framework, along with Tailwind CSS for the frontend and primarily Typescript for the backend processes. The ML-KEM and ML-DSA algorithms are implemented using a modification of an existing Javascript library 'pqc' [16] that allows for the display of the inner workings of both algorithms as opposed to only the outputs. The LLL and BKZ algorithms are implemented through a modification of lll-attack-runner [9] that allows for the display of textual information relating to the algorithms, are visualized using a combination of D3 and React.

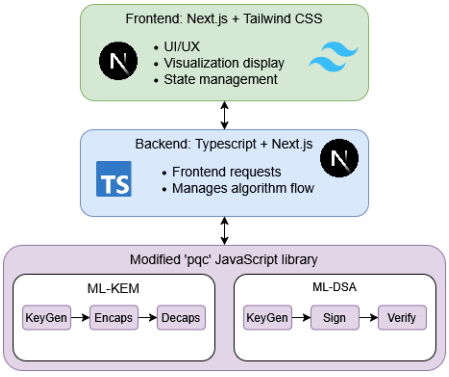


Fig. 20: Architecture of Latviz's ML-KEM and ML-DSA visualization

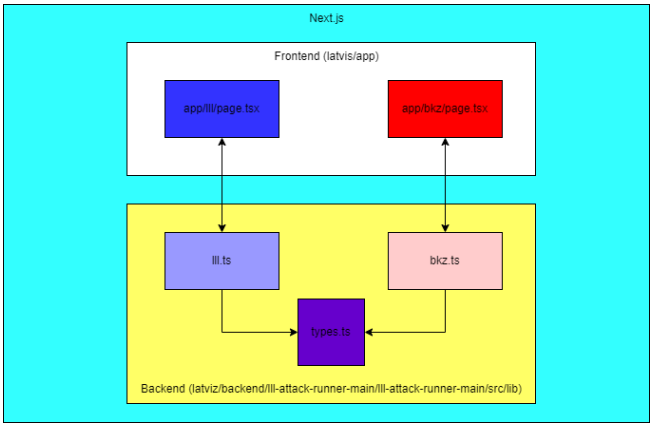


Fig. 21: Program structure of Latviz's LLL and BKZ visualization

3.2 Visualization Structure

3.2.1 ML-KEM and ML-DSA

The visualizations for both algorithms follow the same general structure. Both contain three main sections, starting with an about section at the top of the page, followed by a visualization of the scheme in use, and finally by a full visualization of an instance of the chosen algorithm. For the sake of brevity, this section will only be going into the page for ML-DSA, though both pages are structurally identical.

After opening the page for the algorithm, the first thing the user sees is a short explanation on the chosen algorithm; what it is, what it is set to replace, and what its difficulty is based on. Below is what the section looks like for ML-DSA:

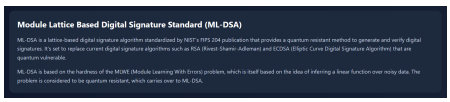


Fig. 22: ML-DSA about component

ML-DSA is described in terms that should be familar to most people with an awareness of cryptography. Rather than focusing on the underlying lattice mathematics, the description emphasizes practical concepts such as signature generation, signature verification, quantum resistant security, and the relationship between ML-DSA and existing algorithms like RSA and ECDSA. This keeps the introduction accessible while preserving the key ideas needed to understand its role in modern cryptographic systems.

After that is a section that briefly explains each stage of the chosen algorithm and provides an animation showing the scheme in use.

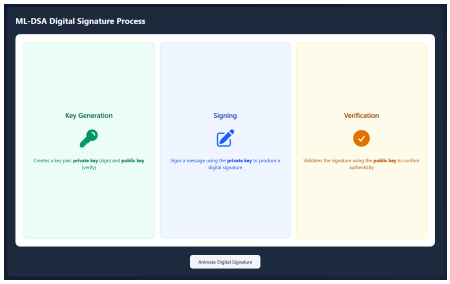


Fig. 23: ML-DSA Process

The section first shows a short description of the purpose of each stage of either algorithm. For ML-DSA, this is key generation, signing, and verification. If the user clicks the button on the bottom of the section, an animation will play that shows the scheme being used in a simplified abstraction of a real world example.

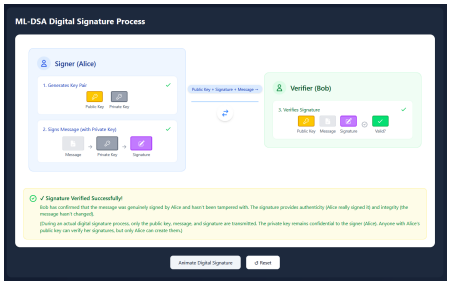


Fig. 24: ML-DSA Process Animation

Below that is a section that displays the an instance of the algorithm in full, including the variables and functions involved in its inner workings. Within it are four different components that each have different functions.

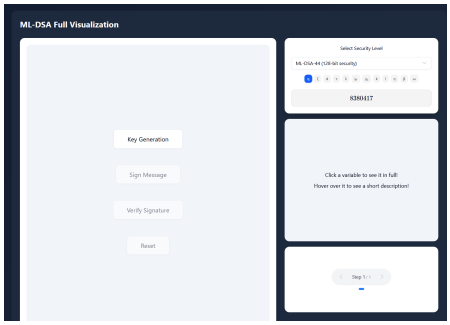


Fig. 25: ML-DSA Full Animation Section

Starting from the top-right is the security level selection component. The user can pick between the security levels ML-DSA-44, ML-DSA-65, and ML-DSA-87, though it defaults to the lowest security level. Beneath the selection, the parameters of the chosen security level are displayed. Whenever a security level is chosen, the visualization is reset and a new instance of ML-DSA is created according to the new security level.

(a) Security level selection (b) Parameters per security level

Fig. 26: Security Level Selection Component

The leftmost component is the main part of this section. It contains four buttons. The first three lead to visualizations of the three stages of ML-DSA. The visualizations come in the form of step-by-step abstractions of the main algorithms of each stage, complete with explanations on the relevant variables and functions.

In ML-DSA specifically, there is an input section for the signing stage that only appears after the key generation stage has been gone through. It needs to be filled in before the signing stage can be accessed, and it is the only actual input from the user that can be used for running an instance of ML-DSA aside from security level selection.

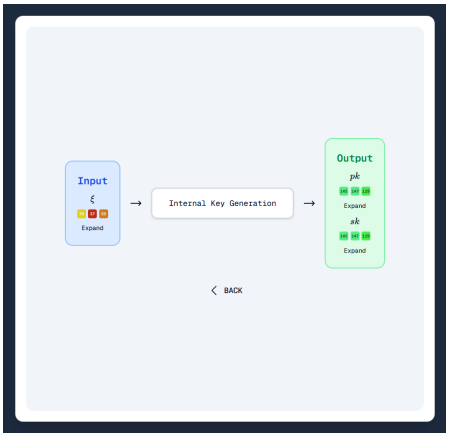
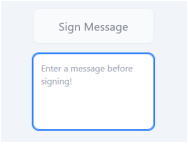
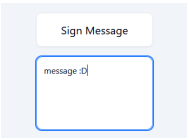


Fig. 27: ML-DSA Full Animation Section



(a) Empty message input

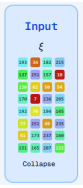


(b) Filled message input

Fig. 28: Input for Sign Message

As interactivity forces engagement and engagement leads to more effective learning [15], these visualizations are interactive. Each variable is displayed in the form of a colored matrix which can be expanded on click to show more values. When hovered on, a popup will appear beside it that describes what it is and provides extra details relating to what it is used for or how it is calculated.

(a) Unexpanded variable



(b) Expanded variable

(c) Popup description

Fig. 29: Variables in the form of colored matrices

Additionally, when a variable is clicked, the component in the middle of the right column will show that variable in scrollable form. This allows for the entire variable to be visible instead of only the first 32 values. There is also an option for the variable to be visible in array form for easier viewing of the values.

(a) Grid view (b) Array view

Fig. 30: Full variables in the side component

When a function is clicked on, it will lead to a visualization of that function. For example, the internal key generation function seen in Figure 25 (more formally known as ML-DSA.KeyGen_internal) will show the following:

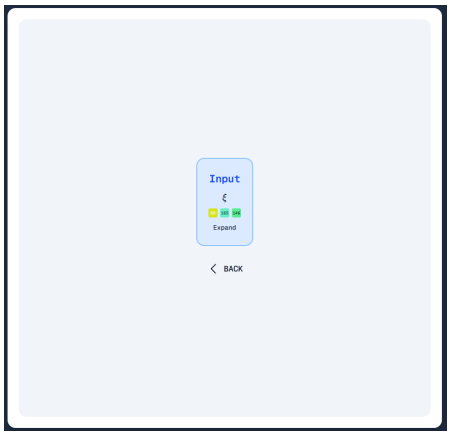


Fig. 31: ML-DSA Full Animation Section (Step 1)

The figure above shows the first step of the function, the input. Subsequent steps can be shown and removed by clicking on the right and left arrows on the lower-right component. Instead of showing the entire function at once, this serves to split the function into more easily followed steps where one part clearly leads to another.

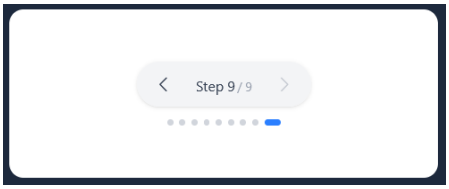


Fig. 32: Stepping Component

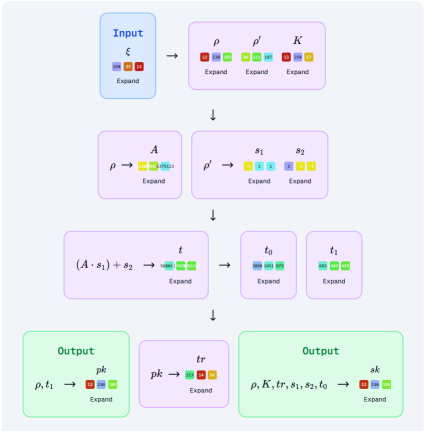


Fig. 33: ML-DSA Full Animation Section (Step 9)

For ML-DSA specifically, there is also a loop function that visualizes each iteration of the valid signature generation loop, elaborating on where and why each specific iteration prior to the last one is invalid.

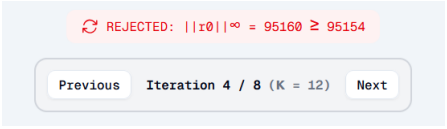


Fig. 34: Sign Loop Iteration Control

3.2.2 LLL and BKZ

Vector visualization is a very essential concept for gaining proper understanding of linear algebra concepts [11]. This also includes LLL and BKZ algorithms. And so, for both 2D and 3D visualization panels in each modules of LLL and

BKZ, we will depict the vectors extracted from the input (the matrix we inserted) as Euclidean vectors within 2D and 3D Cartesian coordinate system. The reason why the Euclidean vectors were chosen to represent the processes as well the inputs of the LLL and BKZ is because both algorithms are obviously lattice reduction algorithms, which operate on the vectors within a Euclidean space. As such, visualizing both algorithms through Euclidean vector representations become a natural choice to show the behavior of lattice bases during the lattice reduction processes. Subprocesses within LLL and BKZ such as the Gram-Schmidt Orthogonalization, basis reduction, and vector swapping (LLL), as well the localized LLLs (BKZ) are involving changes of length, direction, and angles of these vectors. To numerically map the Euclidean Space, the Cartesian coordinate system is implemented, on the basis that lattice basis vectors are naturally represented numerically as coordinates in Euclidean space. Since the LLL and BKZ algorithms operate through geometric transformations of vectors, it could be stated that the Cartesian coordinate system is an intuitive way to visually display the changes of the properties of these vectors.

Within the Cartesian coordinate system used, whether it is the 2D or the 3D visualization, these vectors are depicted as arrows that are anchored to the origin point of (0,0) in 2D Cartesian coordinate and (0,0,0) in 3D Cartesian coordinate. The reason why the coordinates of (0,0) and (0,0,0) become common origin points of all vectors within the visualization panel is to simplify the observation of geometric properties between basis vectors. Thus, by anchoring all vectors to these common origin points, users can directly compare the orientations and magnitudes of vectors without additional coordinate transformations or positional offsets. Conventionally, how the vectors are visualized in linear algebra and vector geometry in the linear algebra (which also includes lattice reduction algorithms like LLL and BKZ) are as arrows originating from the coordinate origin, which in this case, is (0,0) for 2D visualization and (0,0,0) for 3D visualization. Each arrows has different colouring scheme and is unique in terms of value, as every vectors are unique (as they're linearly independent). No vectors would have more than one arrow that represents them (as they all must be linearly independent). And thus, by representing basis vectors as Euclidean vectors inside Cartesian coordinate systems which all starts from (0,0) as the "anchor point", and with unique colour for each single vector, the users are expected to be able to visually observe how highly non-orthogonal bases gradually transform into shorter and more orthogonal bases throughout the LLL and BKZ algorithms in an easier way.

Inside the visualization modules for LLL and BKZ, some elements from both algorithms are better visualized, while some others are better shown in textual format.

The whole visualization process begins after the user insert their matrix in the input field, and then clicked "Insert Matrix" button. Within the frontend system, the inserted matrix is parsed into a 2D array, which represents the basis of the inserted matrix. The outer array represents the entire matrix, while

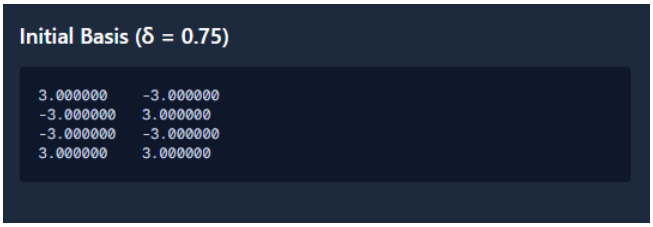


Fig. 35: The initial basis panel (LLL)

the inner array represents the individual basis vectors which are represented by spatial Cartesian coordinates. These basis vectors will be shown as vectors arranged vertically from top to bottom (just like their matrix counterparts) with the first vector is the topmost vector basis. For the LLL visualization module, the 2D array, alongside with the inserted δ-value will be passed into the runLLL function (which executes a LLL process) and runBKZ function (which executes a BKZ process) in lll.ts and in bkz.ts. For the BKZ visualization module, the 2D array, the δ-value, and the β-value will be passed into the runBKZ function in bkz.ts, which executes a BKZ process.

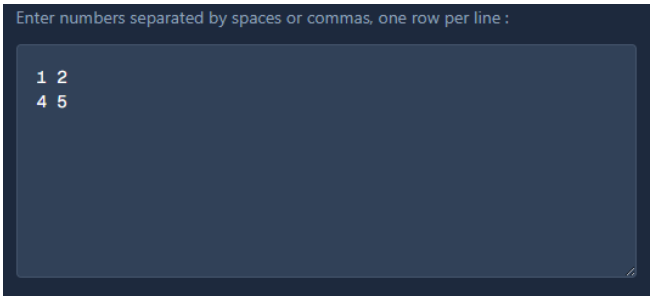


Fig. 36: The input matrix

As what was already stated before, the basis vectors extracted from the input matrix are represented visually as Euclidean vectors inside Cartesian coordinate systems. Thus, each vector is displayed as an arrow originating from the coordinate origin ((0,0) in the 2D visualization or (0,0,0) in the 3D visualization), each one having a unique color, and each one pointing toward the coordinates which represent their numerical values - or the elements of the matrix. For example, the vector $v_1 = (4, 5)$ is visualized as an arrow from the origin point of (0,0) toward the point (4,5). Because all vectors are anchored to the origin point, this allows users to observe the length and direction of these vectors, as well the change of them during the reduction process from the beginning until the final output.

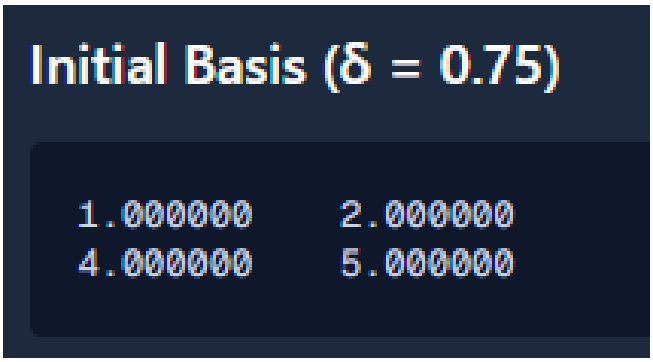


Fig. 37: Basis vectors from the input matrix

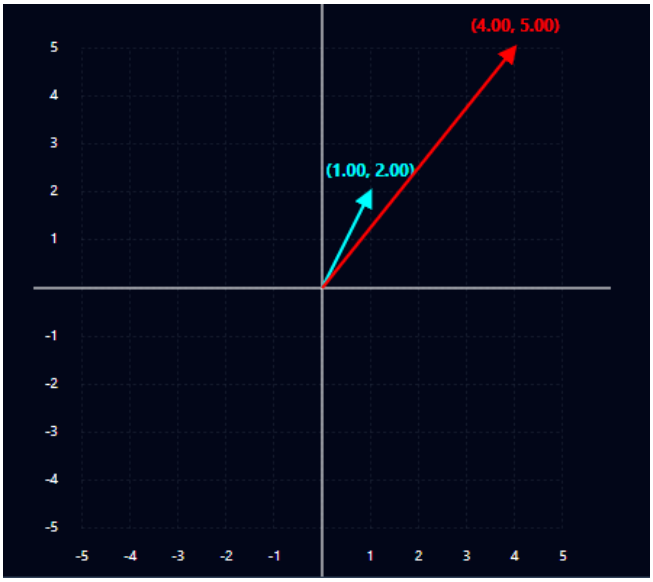


Fig. 38: Basis vectors from the input matrix, represented in the Cartesian coordinate within a Euclidean space

The Gram-Schmidt Orthogonalization subprocess would be better off explained through textual explanation in a pop-up window when the user clicks "Show Calculations" button in the "Step and Calculation Details" panel (which we will explain it in the section 4.3.3). At first, the LLL module's frontend source code (lll/page.tsx) would call the runLLL function from the backend source code of lll.ts and inserts the needed parameters - which consist of the basis vectors, the δ-value, and the Boolean flag "true". Within the runLLLL function, it would call the function gramSchmidt and the program would insert the basis vectors to the function and the gramSchmidt function would perform a Gram-Schmidt Orthogonalization to these vectors. The function returns the orthogonalized version of these vectors. Later, the function generateGSOCalculations would be called and then these orthogonalized vectors, alongside the original vectors, are inserted to the generateGSOCalculations function in order to generate a textual display of the mathematical steps behind the Gram-Schmidt Orthogonalization.

Next, there is vector reduction process, which is shown in the form of a textual explanation of its calculation steps in a pop-up window. How the textual explanations of the calculations behind the vector reduction and vector swap processes are displayed is roughly similar to the Gram-Schmidt process explanation. When the frontend called runLLL function from the backend source code of lll.ts, after performing Gram-Schmidt Orthonalization and other conditions for the vector reduction to be performed, it would call the function vectorSubtract - which as its name suggests, performs a vector size reduction process. The output produced by vectorSubtract, together with the previous vector and the currently examined vector would then be inserted into a function named generateReduceCalculations in order to generate a textual display of the mathematical steps behind the vector size reduction process.

Then, if the current basis doesn't satisfy the Lovasz Condition, it wouldn't only triggered a vector reduction, but also the vector swap after the vector reduction. The currently examined vector would has its index getting switched with the index of the previous vector. Later, the function of generateSwapCalculations would be executed, which generates a textual display of the mathematical steps behind the vector swap process.

And then, there is Lovasz Condition. It is depicted as a status panel of two states ("Satisfied" and "Not Satisfied"). Unlike with the usual mechanism of Lovasz Condition checking, for convenience reasons, the Lovasz Condition status is continously being displayed even when the current process isn't the Lovasz Condition checking as in the common LLL algorithm. This is achieved through the addition of the the function computeLovaszStatus in the app/lll/page.tsx functions to continously checking the Lovasz Condition status in order to keep the Lovasz Status panel has a status at one time, from the first step of the process until the final step. The regular Lovasz Condition checker - the function lovaszCondition in the backend code - was created to perform Lovasz Condition checking only after the currently examined vector has been reduced. Of course, there is a risk of these two Lovasz Condition checkers would have two different answers at the same time, but this risk had been eliminated by employing

them into separate responsibilities. The computeLovaszStatus rechecks the Lovasz Status condition based on the existing data gained from LLL execution by the backend.

And finally, the SVP. The SVP is computed throughout each of the BKZ iterations, and in LLL, computed from the vectors from the final output. However, Latviz would only show the SVP at the final step of both algorithms, and it is displayed within a dedicated panel in the LLL and BKZ modules.

4 Evaluation and Results

This section describes the method used to evaluate the effectiveness of Latviz as a visualization tool for ML-KEM, ML-DSA, LLL, and BKZ as well as the results of the evaluation.

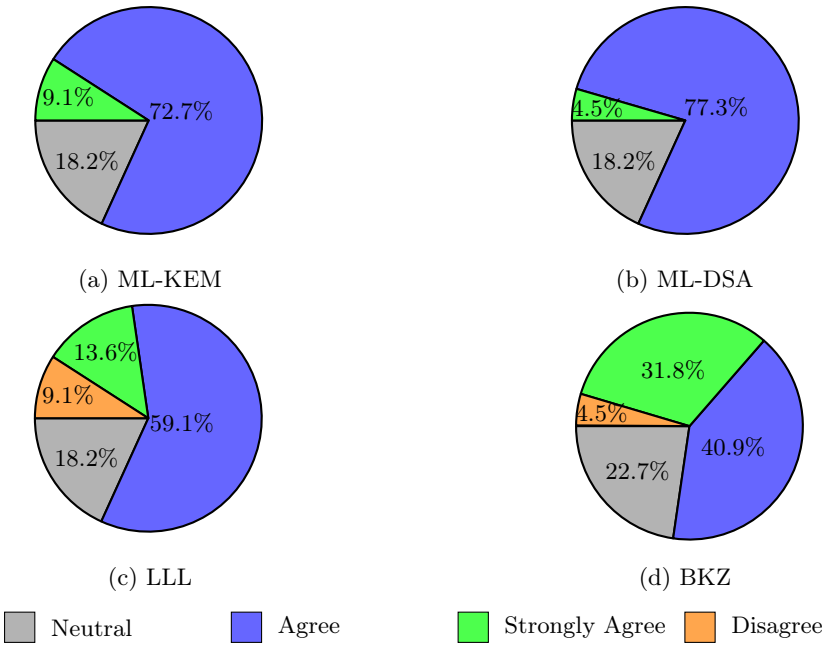
4.1 Methods

To evaluate the effectiveness of Latviz as a visualization tool for ML-KEM, ML-DSA, LLL, and BKZ, a user survey of Latviz users was conducted. The aim of the survey was to determine whether the tool helped users in their understanding of the four algorithms, if it was effective in doing so, and which parts were found to be adequate or not for the learning process. The group of respondents consisted of 22 university students chosen through convenience sampling, with varying degrees of understanding relating to cryptography, post-quantum cryptography, and lattice-based cryptography prior to using Latviz.

The survey consists primarily of 5-point Likert-scale questions [19] ranging from 'Strongly Agree' to 'Strongly Disagree' and focuses on evaluating the following aspects of the visualization: overall understanding of the algorithms, clarity and ease of understanding of the explanations, the effectiveness of the step-by-step visualization approach, comprehension of variables and internal algorithmic processes, the usability and readability of the interface, and the perceived effectiveness of Latviz as a learning tool. In addition, an optional section was added to gather user suggestions or feedback that was not covered by the available questions.

4.2 Results

On the whole, the results of the user survey suggest that Latviz did improve users' understanding of ML-KEM, ML-DSA, LLL, and BKZ, and was also considered an effective tool for learning the algorithms.



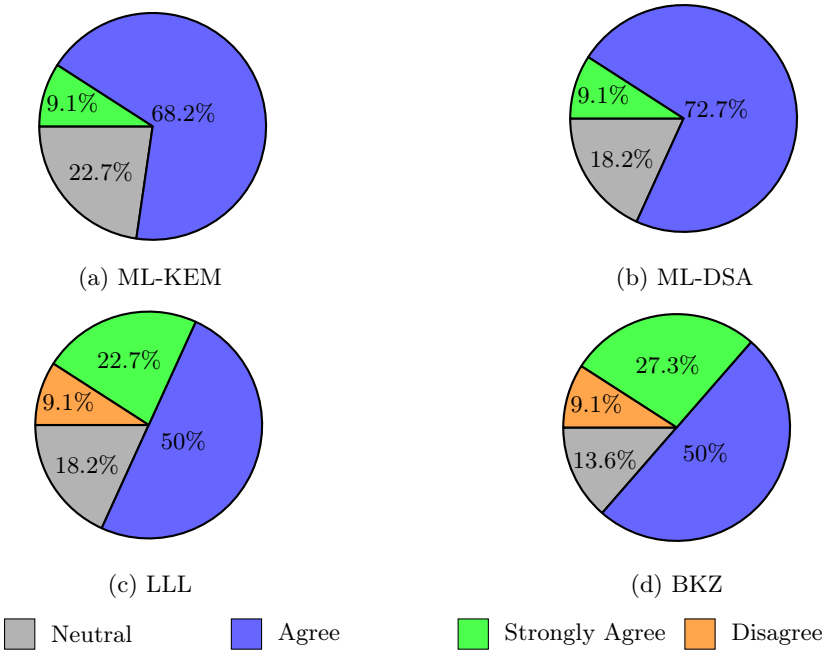
(a) ML-KEM

(b) ML-DSA

(c) LLL

(d) BKZ

Fig. 39: Responses regarding improved understanding after using Latviz.



(a) ML-KEM (b) ML-DSA

(c) LLL (d) BKZ

Fig. 40: Responses regarding whether Latviz was perceived as an effective learning tool.

To examine whether these perceptions differed based on participants' prior experience with cryptography, respondents were grouped based on their self-reported familiarity with cryptography and post-quantum cryptography. The groups were defined as NC (No Familiarity with Cryptography), LC (Low Familiarity with Cryptography), YC-YPQ (Familiar with Cryptography and Post-Quantum Cryptography), and YC-LPQ (Familiar with Cryptography but Low Post-Quantum Cryptography Knowledge).

Table 3: Respondent Groups

Group	Amount of Respondents
NC	9
LC	4
YC-YPQ	6
YC-LPQ	3

Following the classification of respondents into groups, the overall survey responses were analyzed. To start with, the table 4 presents the distribution of responses for the ML-KEM section of the survey.

Table 4: Responses to ML-KEM Evaluation Questions

Statements	SD	D	N	A	SA
After using Latviz, I feel I better understand ML-KEM.	0%	0%	18.2%	72.7%	9.1%
The step-by-step visualization helped me understand the overall flow of ML-KEM.	0%	0%	22.7%	63.6%	13.6%
The explanations provided were easy to understand.	0%	0%	40.9%	54.5%	4.5%
I was able to understand the functions and variables used in ML-KEM.	0%	0%	31.8%	63.6%	4.5%
The visualization helped me understand the relationships between processes within the algorithm.	0%	4.5%	13.6%	50%	31.8%
I was able to follow changes to data and variables throughout the algorithm.	0%	4.5%	18.2%	59.1%	18.2%
I was able to understand how inputs were processed into outputs.	0%	0%	27.3%	59.1%	13.6%
The visualization was clear and easy to follow.	0%	4.5%	9.1%	54.5%	31.8%
Latviz is an effective learning tool for ML-KEM.	0%	0%	22.7%	68.2%	9.1%

As mentioned before, the overall responses regarding ML-KEM were largely positive. 72.7% of users agreed with the statement that Latviz deepened their understanding of ML-KEM, 9.1% strongly agreed, while 18.2% that reported that they did not gain further understanding. However, only 68.2% of users reported that they agreed with the statement that Latviz is an effective learning tool for ML-KEM, with the percentage of users that strongly agreed staying at 9.1%, followed by 22.7% that were neutral. Taking a closer look at the respondents, it appears that despite personally deepening their own understanding of ML-KEM, one respondent in group LC and another in group NC felt neutral about Latviz being an effective learning tool for ML-KEM.

To better understand the factors contributing to these overall perceptions, respondents were also asked to evaluate specific features of the ML-KEM visualization.

Overall, responses to these questions were positive, with most participants agreeing that the visualization helped them understand the internal workings of the algorithm. The highest levels of agreement were observed for statements regarding the relationships between algorithmic processes and the clarity of the visualization itself. In both cases, over 80% of respondents agreed or strongly agreed that the visualizations helped them understand how different components of ML-KEM interact and that the interface was clear and easy to follow.

It should be noted however, that 4.5% of respondents disagreed in both cases. Those disagreements, along with a majority of the neutral response, came from the NC group of respondents. This suggests that those without prior knowledge in cryptography may have found it more difficult to interpret the relationships between algorithmic processes and follow the visualizations than respondents with greater familiarity with the subject.

Similarly, a large majority of respondents reported being able to follow changes in data and variables throughout the execution of the algorithm and understand how inputs were transformed into outputs. These findings suggest that the step-by-step visualization approach was effective in illustrating the flow of information through the algorithm.

The least positive responses were observed for the clarity of the explanations provided. While no respondents disagreed that the explanations were easy to understand, 40.9% selected a neutral response, indicating that a substantial portion of users felt that the written explanations could be improved. Likewise, 31.8% of respondents were neutral regarding their understanding of the functions and variables used within ML-KEM. Taken together, these findings suggest that although the visualizations themselves were generally effective, some users may benefit from explanations that assume less prior knowledge and provide additional context for unfamiliar concepts.

Next, table 5 presents the distribution of results for the ML-DSA section of the survey.

Table 5: Responses to ML-DSA Evaluation Questions

Question	SD	D	N	A	SA
After using Latviz, I feel I better understand ML-DSA.	0%	0%	18.2%	77.3%	4.5%
The step-by-step visualization helped me understand the overall flow of ML-DSA.	0%	4.5%	18.2%	59.1%	18.2%
The explanations provided were easy to understand.	0%	4.5%	27.3%	54.5%	13.6%
I was able to understand the functions and variables used in ML-DSA.	0%	13.6%	9.1%	63.6%	13.6%
The visualization helped me understand the relationships between processes within the algorithm.	0%	0%	9.1%	59.1%	31.8%
I was able to follow changes to data and variables throughout the algorithm.	0%	0%	18.2%	72.7%	9.1%
I was able to understand how inputs were processed into outputs.	0%	4.5%	13.6%	59.1%	22.7%
The visualization was clear and easy to follow.	0%	0%	18.2%	59.1%	22.7%
Latviz is an effective learning tool for ML-DSA.	0%	0%	18.2%	72.7%	9.1%

Similar to the ML-KEM results, the responses regarding ML-DSA were largely positive. A total of 81.8% of respondents agreed or strongly agreed that Latviz

improved their understanding of ML-DSA, while the same portion considered it an effective learning tool for the algorithm. Furthermore, no respondents disagreed with either statement.

For the individual components of the visualization, the strongest responses were observed for the statement regarding the relationships between algorithmic processes, with 90.9% of respondents agreeing or strongly agreeing that the visualization helped them understand how different parts of ML-DSA interact. Similarly, 81.8% of respondents reported that they were able to follow changes in data and variables throughout the execution of the algorithm, and an equal percentage found the visualization clear and easy to follow.

The least positive responses were associated with understanding the functions and variables used within ML-DSA. While 77.2% of respondents agreed or strongly agreed that they understood these elements, 13.6% disagreed and 9.1% selected a neutral response. Two of those disagreements and the entirety of the neutral response came from the NC group, with only one disagreement coming from the YC-LPQ group.

Similarly, the explanations provided received a relatively high proportion of neutral responses (27.3%) and one disagreement. Of the six neutral responses, four came from the NC group, one from the LC group, and one from the YC-LPQ group. The sole disagreement also originated from the YC-LPQ group. These results suggest that respondents with limited cryptographic knowledge, as well as those familiar with cryptography but less familiar with post-quantum cryptography, found the written explanations less accessible than other aspects of the visualization.

Overall, these findings indicate that while the visualizations were generally effective in conveying the structure and flow of ML-DSA, some users may have benefited from explanations that assumed less background knowledge and provided additional context for unfamiliar concepts.

The third table below presents the results of the LLL section of this survey.

Table 6: Responses to LLL Evaluation Questions

Statements	SD	D	N	A	SA
After using Latviz, I think I understand LLL better than before	0%	9.1%	18.2%	59.1%	13.6%
The step-by-step visualization with "Back" and "Next" buttons to control the process flow helps me to understand the flow of LLL	0%	4.5%	18.2%	50%	27.3%
It's easy to understand explanations shown in the "Calculation Details"	0%	9.1%	9.1%	45.5%	36.4%
The "Lovasz Status" and "Step" Panels help me enough to understand the ongoing process in LLL	0%	4.5%	18.2%	50%	27.3%
I understand the roles of the variables used in LLL	0%	18.2%	22.7%	36.4%	22.7%
I can track the changes in data or variables at every step of LLL	0%	9.1%	18.2%	45.5%	27.3%
I understand how the input is processed to produce the output	4.5%	4.5%	13.6%	45.5%	31.8%
The visual display is clear and not confusing	0%	9.1%	18.2%	50%	22.7%
Latviz is an effective learning tool for LLL.	0%	9.1%	18.2%	50%	22.7%

And finally, the last table below shows the results of the BKZ section of this survey.

Table 7: Responses to BKZ Evaluation Questions

Statements	SD	D	N	A	SA
After using Latviz, I think I understand BKZ better than before	0%	4.5%	22.7%	40.9%	31.8%
The step-by-step visualization with "Back" and "Next" buttons to control the process flow helps me to understand the flow of BKZ	0%	4.5%	13.6%	36.4%	45.5%
It's easy to understand explanations shown in the "Calculation Details"	0%	9.1%	13.6%	45.5%	31.8%
I understand the roles of the variables used in BKZ	0%	13.6%	18.2%	50%	18.2%
I can track the changes in data or variables at every step of BKZ	0%	9.1%	13.6%	45.5%	31.8%
I understand how the input is processed to produce the output	0%	9.1%	9.1%	59.1%	22.7%
The visual display is clear and not confusing	0%	4.5%	13.6%	59.1%	22.7%
Latviz is an effective learning tool for BKZ.	0%	9.1%	13.6%	50%	27.3%

72.7% of respondents agreed or strongly agreed that Latviz increased their understanding of LLL and BKZ. Of those respondents, a majority of the neutral and disagree responses are of the NC group, with only two respondents from YC-

LPQ for LLL and one each in YC-LPQ and YC-YPQ for BKZ. Like the results of ML-KEM and ML-DSA, this suggests that users with less prior knowledge of cryptography might have difficulties with understanding the learning tool.

Of all UI elements surveyed for both the LLL and BKZ visualization modules, the most positive responses were for the calculation details panel for LLL (81.9%), and the step through iteration section. for BKZ (81.9%). The less positive responses were associated with the roles of the variables used in both algorithms. In particular, only 59.1% of all respondents understood the roles of the variables used in the LLL algorithm, and for BKZ, it is slightly lower at just 58.2%. This suggests that Latviz is not yet fully effective at explaining the variables involved in these algorithms.

5 Conclusion

This paper presents the digital visualization tool Latviz, which was created to help learners more easily understand the entire process behind the ML-KEM and ML-DSA cryptographic standards and the lattice basis reduction algorithms LLL and BKZ. As post-quantum cryptography becomes increasingly important, there is a growing need for educational resources that allow learners to understand not only the purpose of these algorithms but how they work. Existing resources often focus on high-level overviews or require substantial prior knowledge of cryptography. To address this gap, Latviz was developed to provide complete step-by-step visualizations for ML-KEM and ML-DSA, accompanied by supplementary explanations of the functions, variables, and inner processes involved in their execution, and algorithm tracing complete with explanations for LLL and BKZ.

The resulting tool enables users to explore all four algorithms interactively, inspect intermediate values and operations, and follow the transformation of inputs into outputs throughout the execution process. By combining detailed visualizations with explanatory content, Latviz aims to make complex post-quantum cryptographic concepts more accessible to learners with varying levels of prior cryptographical knowledge.

To evaluate the effectiveness of the tool, a survey involving 22 participants sampled through convenience sampling with differing levels of familiarity with cryptography and post-quantum cryptography was conducted. The results indicate that Latviz was generally perceived as an effective learning tool for both algorithms, with 77.3%, 81.8%, 72.7%, and 77.3% of respondents considering it effective for ML-KEM, ML-DSA, LLL, and BKZ respectively. Furthermore, 81.8% of respondents reported that the tool improved their understanding of ML-KEM and ML-DSA, and 72.7% reported the same for LLL and BKZ. Relating to ML-KEM and ML-DSA, participants responded particularly positively to the step-by-step visualizations and their ability to illustrate the relationships between algorithmic processes. However, the results also suggest that users with limited prior knowledge of cryptography may have trouble with following the provided explanations and may benefit from additional context for unfamiliar

concepts. For LLL and BKZ, participants found the calculation details panel and step-by-step visualizations to be the most helpful, while the sections relegated to explaining the roles of variables were found to be least effective.

Overall, the evaluation results indicate that Latviz serves as an effective tool for improving users' understanding of ML-KEM, ML-DSA, LLL, and BKZ. These findings demonstrate the potential of interactive visualization as an educational approach for helping learners engage with and understand complex post-quantum cryptographic algorithms.

6 Future Work

While the evaluation results indicate that Latviz is effective in supporting users' understanding of ML-KEM, ML-DSA, LLL, and BKZ, there are still several opportunities for improvement. The survey responses suggest that some users, particularly those with limited prior knowledge of cryptography, found certain explanations difficult to follow. Future development could therefore focus on improving the accessibility of the content through the addition of or references to introductory materials, more contextual explanations, and alternative methods of presenting complex concepts.

Future work may also involve expanding the scope of the tool by incorporating additional post-quantum cryptography material such as the NTRU cryptosystem. Additionally, evaluations involving larger and more diverse participant groups as well as pre-test and post-test assessments could provide a more precise and comprehensive measurement of the tool's effectiveness in improving users' understanding of post-quantum cryptographic concepts. Aside from technical improvements, increasing support for different languages can also improve the tool for audiences more comfortable with different languages.

References

1. Avanzi, R., Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J. M., Schwabe, P., Seiler, G., and Stehlé, D. CRYSTALS-Kyber: Algorithm specifications and supporting documentation, 2021. Version 3.02.
2. Bai, S., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schwabe, P., Seiler, G., and Stehlé, D. CRYSTALS-Dilithium: Algorithm specifications and supporting documentation, 2021. Version 3.1.
3. Barker, E., Chen, L., Roginsky, A., Vassilev, A., and Davis, R. Recommendation for the entropy sources used for random bit generation. NIST Special Publication 800-90B, National Institute of Standards and Technology, 2018.
4. Barker, E., Chen, L., Roginsky, A., Vassilev, A., and Davis, R. Recommendation for random bit generator (rbg) constructions. NIST Special Publication 800-90C, National Institute of Standards and Technology, 2025.
5. Barker, E., and Kelsey, J. Recommendation for random number generation using deterministic random bit generators. NIST Special Publication 800-90A Rev. 1, National Institute of Standards and Technology, 2015.

6. Benioff, P. The computer as a physical system: A microscopic quantum mechanical hamiltonian model of computers as represented by turing machines. *Journal of Statistical Physics 22*, 5 (1980), 563–591.
7. Benioff, P. Quantum mechanical hamiltonian models of turing machines. *Journal of Statistical Physics 29*, 3 (1982), 515–546.
8. Bennett, C. H., and Brassard, G. Quantum cryptography: Public key distribution and coin tossing. *Theoretical Computer Science 560* (2014), 7–11. Theoretical Aspects of Quantum Cryptography – celebrating 30 years of BB84.
9. Bennett, Z. R. lll-attack-runner. https://github.com/curiosityz/lll-attack-runner, 2026.
10. Brassard, G. Brief history of quantum cryptography: a personal perspective. In *IEEE Information Theory Workshop on Theory and Practice in Information-Theoretic Security, 2005.* (2005), pp. 19–23.
11. Calderone, D. Vector visualizations, May 2023. Teaching material.
12. development team, T. F. fplll, a lattice reduction library, Version: 5.5.0. Available at https://github.com/fplll/fplll, 2023.
13. Feynman, R. P. Simulating physics with computers. *International Journal of Theoretical Physics 21*, 6 (1982), 467–488.
14. Grout, J., Hanson, I., Johnson, S., Kramer, A., Novoseltsev, A., and Stein, W. Sagemathcell. https://sagecell.sagemath.org, 2011. Version 1.
15. Hundhausen, C. D., Douglas, S. A., and Stasko, J. T. A meta-study of algorithm visualization effectiveness. *Journal of Visual Languages & Computing 13*, 3 (2002), 259–290.
16. Kalava, N. R., Kukkala, V. R., Kanaparthi, S. S., Gangavarapu, T. V., and Lakshmi, Y. V. Post-quantum security for web applications, 2025. Unpublished manuscript.
17. Kang, D. Ml-kem from scratch. https://pqc.hulryung.com/intro.html, 2026.
18. Li, J., and Nguyen, P. Q. A complete analysis of the bkz lattice reduction algorithm. *Journal of Cryptology 38*, 1 (2025), 12.
19. Likert, R. A technique for the measurement of attitudes. *Archives of Psychology 140* (1932), 1–55.
20. Meyer, A. Post-quantum cryptography: An analysis of code-based and lattice-based cryptosystems, 2025.
21. National Institute of Standards and Technology. Module-lattice-based digital signature standard. Federal Information Processing Standards Publication FIPS 204, National Institute of Standards and Technology, Gaithersburg, MD, USA, Aug. 2024. Published August 13, 2024.
22. National Institute of Standards and Technology. Module-lattice-based key-encapsulation mechanism standard. Federal Information Processing Standards Publication FIPS 203, National Institute of Standards and Technology, Gaithersburg, MD, USA, Aug. 2024. Published August 13, 2024.
23. Shor, P. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th Annual Symposium on Foundations of Computer Science* (1994), pp. 124–134.
24. Silverman, J. H. An introduction to lattices, lattice reduction, and lattice-based cryptography, 2020. IAS/Park City Mathematics Series Graduate Summer School Lecture Notes.
25. Wang, Y., Jin, X., and Liu, J. Learning with errors may remain hard against quantum holographic attacks, 2025.

26. Zhao, Z., and Ding, J. Practical improvements on bkz algorithm. In *Cyber Security, Cryptology, and Machine Learning* (Cham, 2023), S. Dolev, E. Gudes, and P. Paillier, Eds., Springer Nature Switzerland, pp. 273–284.